

Air University



# Digital Logic Design (Lab)

Year 2025

Ms. Faryal Naeem

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: March 10, 2025

**Assignment No. 01****Question 01:**

**Given the Boolean Expression  $Y = \overline{A + B + C}$**

- **Simplify the expression using De Morgan's laws.**
- **Implement the input expression and Simplified expression using only NAND gates.**
- **Verify the output using basic logic gates on Proteus.**

**Solution:****1. Simplification Using De Morgan's Laws:**

**Given:**  $Y = \overline{A + B + C}$

By applying **De Morgan's Law** to the expression at hand, we find:

$$Y = \overline{A + B + C} = \bar{A} \cdot \bar{B} \cdot \bar{C}.$$

**Result:** After simplifying, the expression becomes  $Y = \bar{A} \cdot \bar{B} \cdot \bar{C}$ .

**2. Implementation Using NAND Gates:****a. Original Expression:**  $Y = \overline{A + B + C}$ 

This initial expression is equivalent to a NOR gate. To build it using NAND gates, follow these steps:

**Create NOT Gates (using NANDs):** Use three NAND gates, one for each input (A, B, and C). Tie the inputs of each gate together so that input A goes to the first gate, input B to the second, and input C to the third. This will yield  $\bar{A}$ ,  $\bar{B}$ , and  $\bar{C}$ , respectively.

**Implement NOR (using NAND):** Connect the outputs from the three NOT NAND gates to another NAND gate. To get the desired outcome, invert this output with another NAND gate, effectively performing a NOR operation.

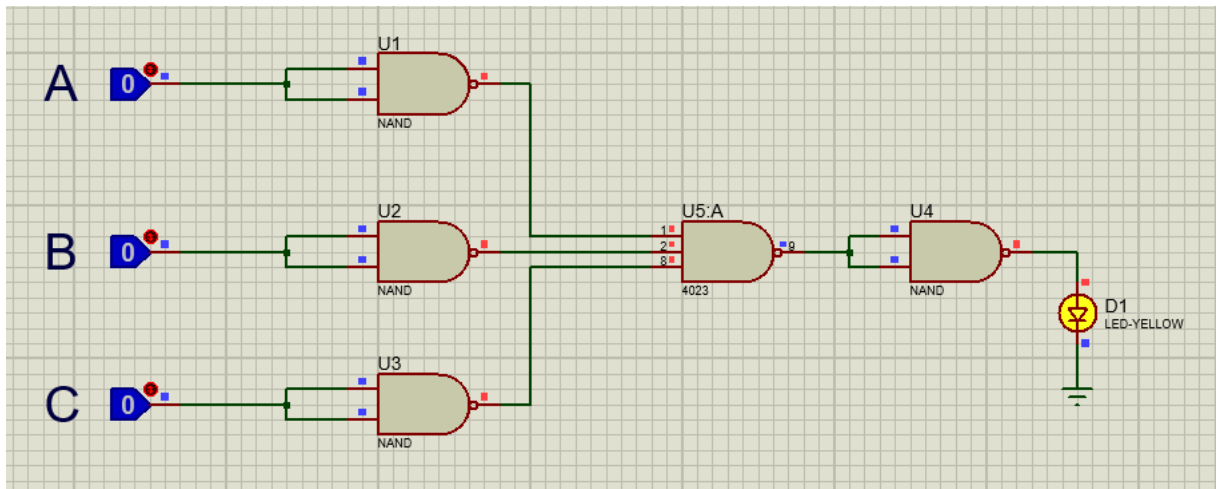
**b. Simplified Expression:**  $Y = \bar{A} \cdot \bar{B} \cdot \bar{C}$ 

**Create NOT Gates (using NANDs):** As before, utilize three NAND gates to generate  $\bar{A}$ ,  $\bar{B}$ , and  $\bar{C}$ .

**Implement AND (using NANDs):** Connect the outputs from the NOT gates into a NAND gate. To achieve the expression  $\bar{A} \cdot \bar{B} \cdot \bar{C}$ , invert this result using another NAND gate to complete the AND operation.

**Input:** A=0, B=0, C=0 → **Output=1**

For all other inputs the output will be 0. For both **Original** and **simplified expressions**.



### 3. Verification on Proteus:

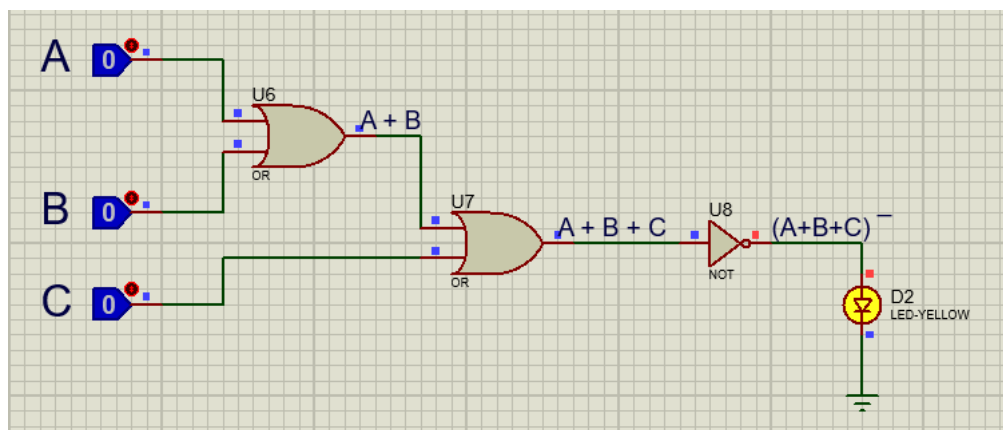
**Truth Table:**

| A | B | C | A+B | A+B+C | $Y = \overline{A+B+C}$ | $\bar{A}$ | $\bar{B}$ | $\bar{C}$ | $\bar{A} \cdot \bar{B}$ | $Y = \bar{A} \cdot \bar{B} \cdot \bar{C}$ |
|---|---|---|-----|-------|------------------------|-----------|-----------|-----------|-------------------------|-------------------------------------------|
| 0 | 0 | 0 | 0   | 0     | 1                      | 1         | 1         | 1         | 1                       | 1                                         |
| 0 | 0 | 1 | 0   | 1     | 0                      | 1         | 1         | 0         | 1                       | 0                                         |
| 0 | 1 | 0 | 1   | 1     | 0                      | 1         | 0         | 1         | 0                       | 0                                         |
| 0 | 1 | 1 | 1   | 1     | 0                      | 1         | 0         | 0         | 0                       | 0                                         |
| 1 | 0 | 0 | 1   | 1     | 0                      | 0         | 1         | 1         | 0                       | 0                                         |
| 1 | 0 | 1 | 1   | 1     | 0                      | 0         | 1         | 0         | 0                       | 0                                         |
| 1 | 1 | 0 | 1   | 1     | 0                      | 0         | 0         | 1         | 0                       | 0                                         |
| 1 | 1 | 1 | 1   | 1     | 0                      | 0         | 0         | 0         | 0                       | 0                                         |

**Using basic Logic gates:**

**a. Original Expression:**  $Y = \overline{A+B+C}$

- In Proteus, set up three input pins labeled A, B, and C.
- Use an OR gate to combine the input of A and B:  $A+B$ .
- Use another OR gate to combine the output from first OR gate and C:  $A+B+C$ .
- Use a NOT gate to get the inverted output:  $\overline{A+B+C}$

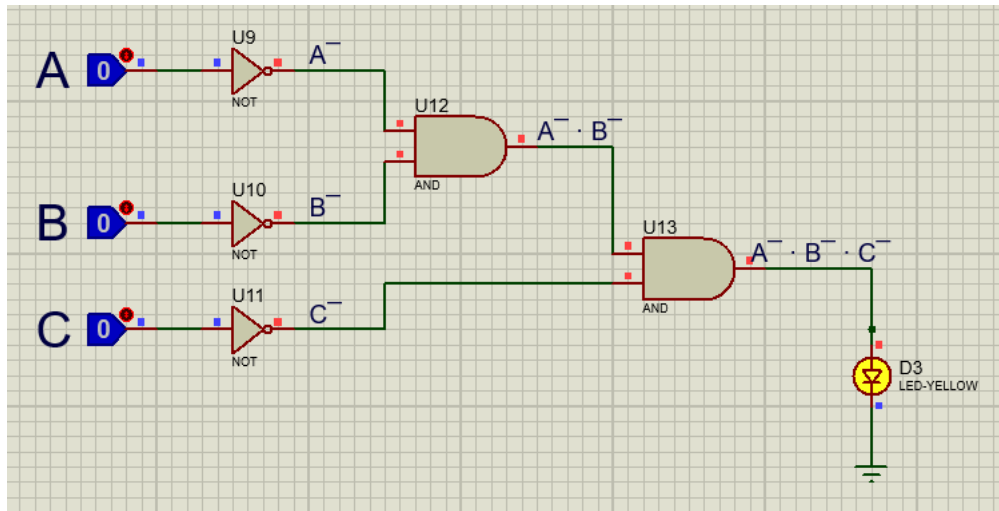


**Input:** A=0, B=0, C=0 → **Output=1**

For all other inputs the output will be **0**.

**b. Simplified Expression:**  $Y = \bar{A} \cdot \bar{B} \cdot \bar{C}$

- In Proteus, place three input pins labeled A, B, and C.
- Use a NOT gate to invert the three inputs:  $\bar{A}$ ,  $\bar{B}$  and  $\bar{C}$ .
- Use an AND gate to combine the input of  $\bar{A}$  and  $\bar{B}$ :  $\bar{A} \cdot \bar{B}$ .
- Use another AND gate to combine the output from first AND gate and  $\bar{C}$ :  $\bar{A} \cdot \bar{B} \cdot \bar{C}$



**Input:** A=0, B=0, C=0 → **Output=1**

For all other inputs the output will be **0**.

**c. Simulation and Truth Table Verification:**

- Tested all 8 possible combinations of inputs (from 000 to 111) for both the “Original” and “Simplified” circuits.
- Recorded the output Y for each input combination.
- Developed a truth table to compare results, organized as follows:
- Ensured that the outputs match across both circuits for every input combination. Thus, confirms their logical equivalence.

### Question 02:

**Implement XOR, XNOR using NAND and NOR gate:**

- Draw the truth table and the Logic Expression
- Attach Simulated results from Proteus

**Solution:****1. XOR (Exclusive OR) Implementation****Logic Expression:**

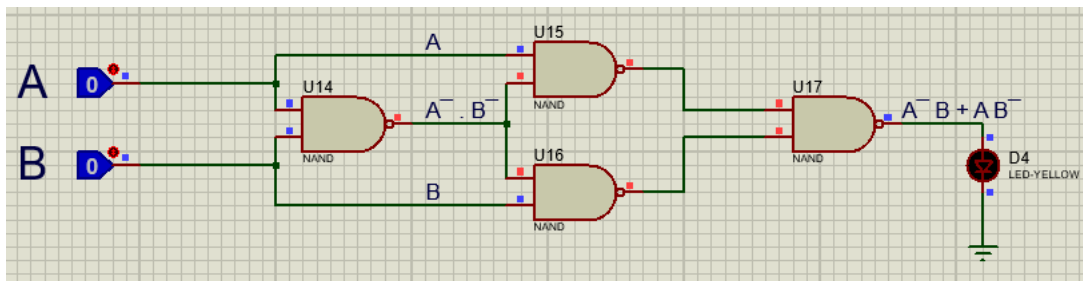
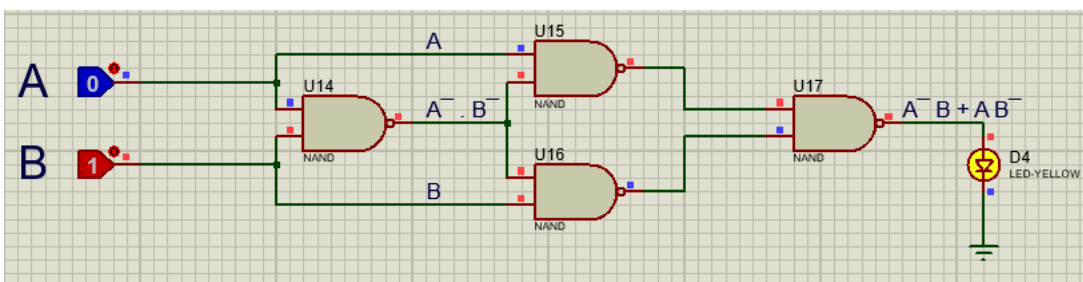
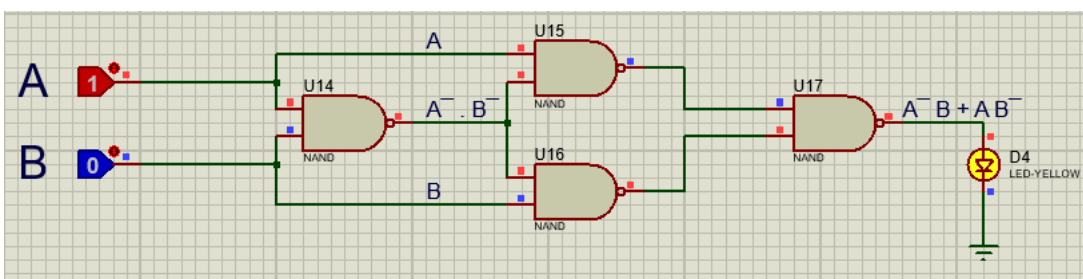
$$Y = A \oplus B = \bar{A}B + A\bar{B}$$

**Truth Table:**

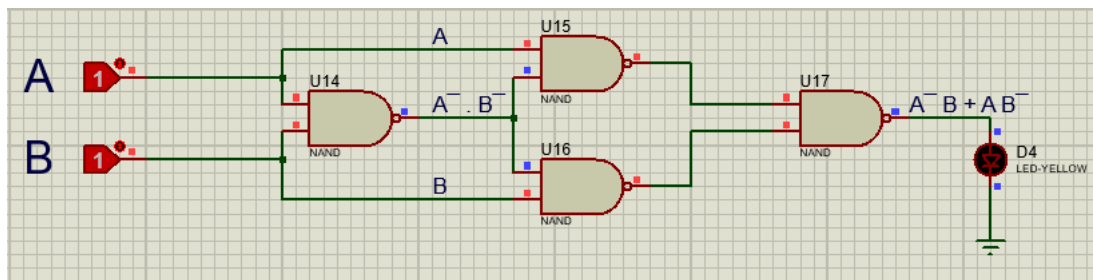
| A | B | $\bar{A}$ | $\bar{B}$ | $\bar{A}B$ | $A\bar{B}$ | $Y = \bar{A}B + A\bar{B}$ |
|---|---|-----------|-----------|------------|------------|---------------------------|
| 0 | 0 | 1         | 1         | 0          | 0          | <b>0</b>                  |
| 0 | 1 | 1         | 0         | 1          | 0          | <b>1</b>                  |
| 1 | 0 | 0         | 1         | 0          | 1          | <b>1</b>                  |
| 1 | 1 | 0         | 0         | 0          | 0          | <b>0</b>                  |

**i. XOR using NAND gate**

- Use a NAND gate to combine A and B.
- Use a second NAND gate to combine output of first NAND gate and A.
- Use a third NAND gate to combine output of first NAND gate and B.
- Use a fourth NAND gate to combine the output of second and third NAND gate.

**Input: A=0, B=0 → Output=0****Input: A=0, B=1 → Output=1****Input: A=1, B=0 → Output=1**

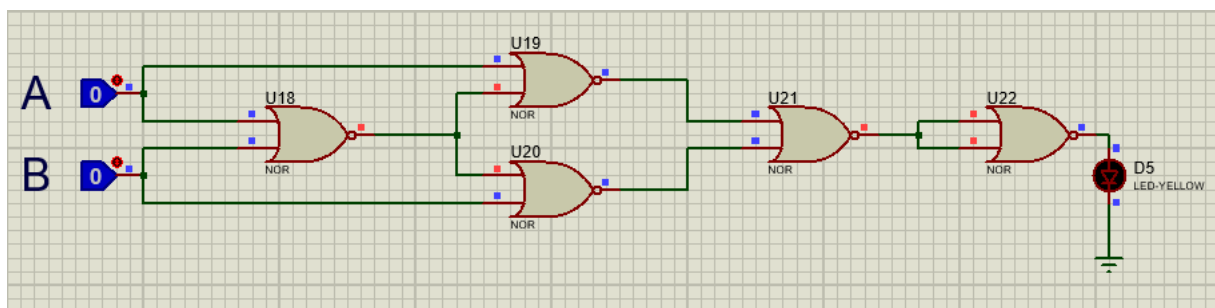
**Input: A=1, B=1 → Output=0**



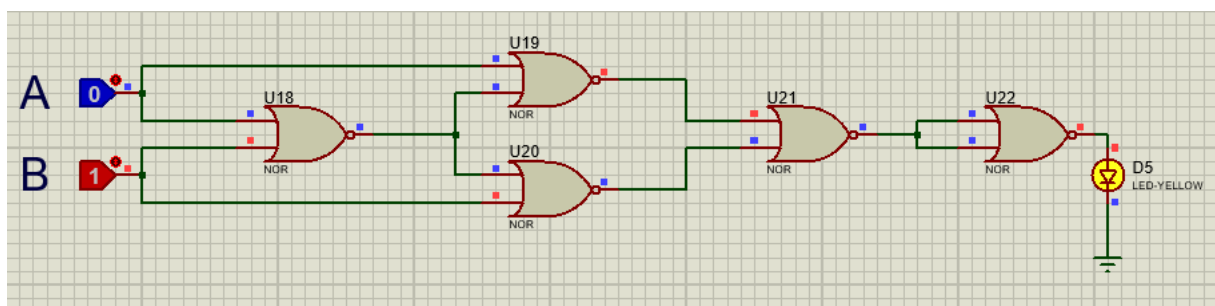
## ii. XOR using NOR gate

- Use a NOR gate to combine the inputs of A and B.
- Use second NOR gate to combine the output of first NOR gate and A.
- Use third NOR gate to combine the output of first NOR gate and B.
- Use fourth NOR gate to combine the output of second and third NOR gate.
- Use fifth NOR gate to invert the final output of fourth NOR gate.

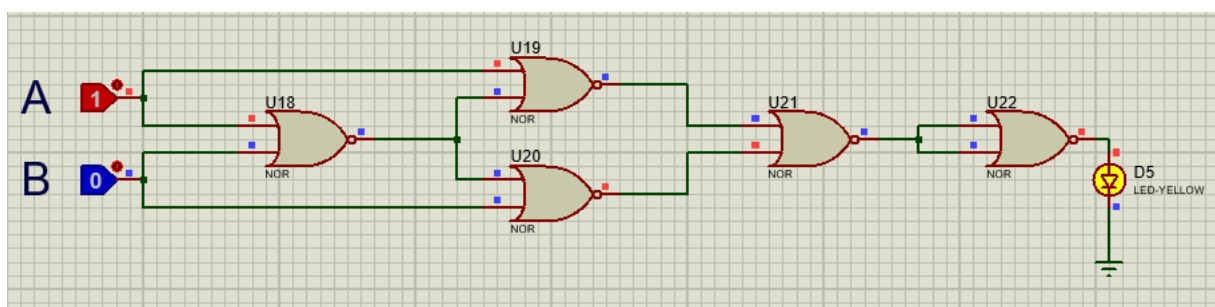
**Input: A=0, B=0 → Output=0**



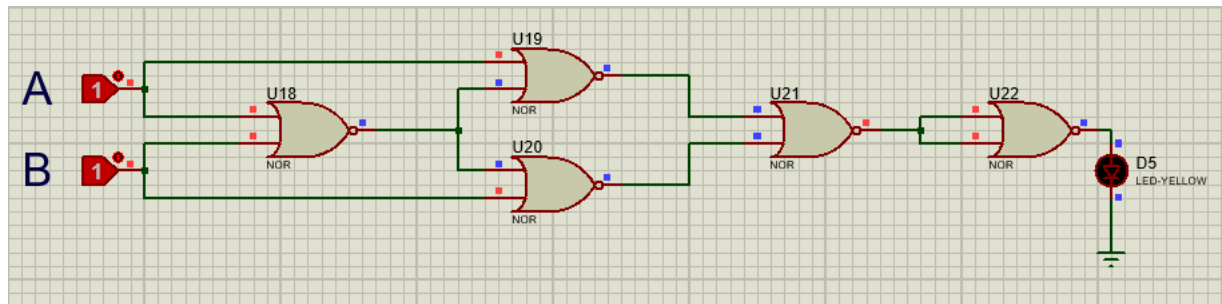
**Input: A=0, B=1 → Output=1**



**Input: A=1, B=0 → Output=1**



**Input: A=1, B=1 → Output=0**



## 2. XNOR (Exclusive NOR) Implementation

**Logic Expression:**

$$Y = A \odot B = \overline{A \oplus B} = AB + \overline{AB}$$

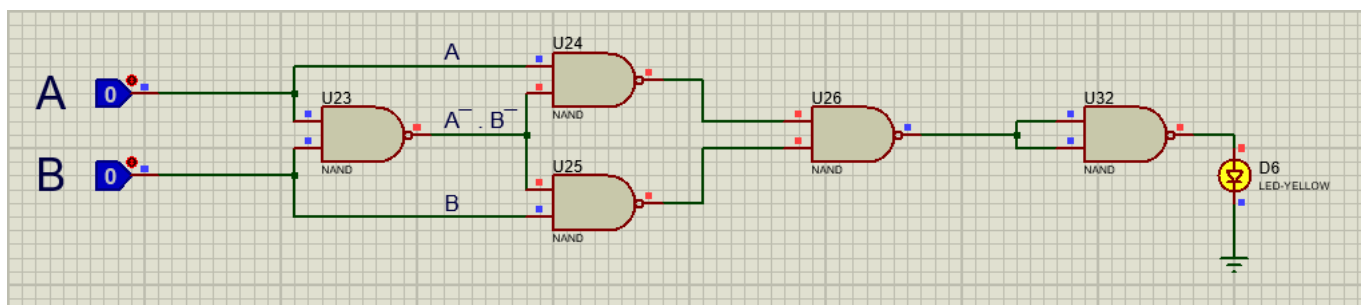
**Truth Table:**

| A | B | $\overline{A}$ | $\overline{B}$ | AB | $\overline{AB}$ | $AB + \overline{AB}$ |
|---|---|----------------|----------------|----|-----------------|----------------------|
| 0 | 0 | 1              | 1              | 0  | 1               | <b>1</b>             |
| 0 | 1 | 1              | 0              | 0  | 0               | <b>0</b>             |
| 1 | 0 | 0              | 1              | 0  | 0               | <b>0</b>             |
| 1 | 1 | 0              | 0              | 1  | 0               | <b>1</b>             |

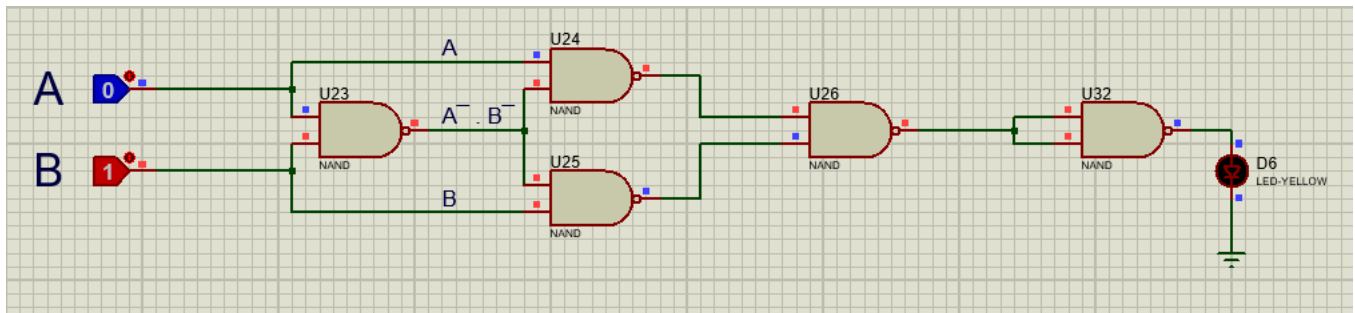
### i. XNOR using NAND gate

- Use a NAND gate to combine A and B.
- Use a second NAND gate to combine output of first NAND gate and A.
- Use a third NAND gate to combine output of first NAND gate and B.
- Use a fourth NAND gate to combine the output of second and third NAND gate.
- Use a fifth NAND gate to invert the final output of fourth NAND gate.

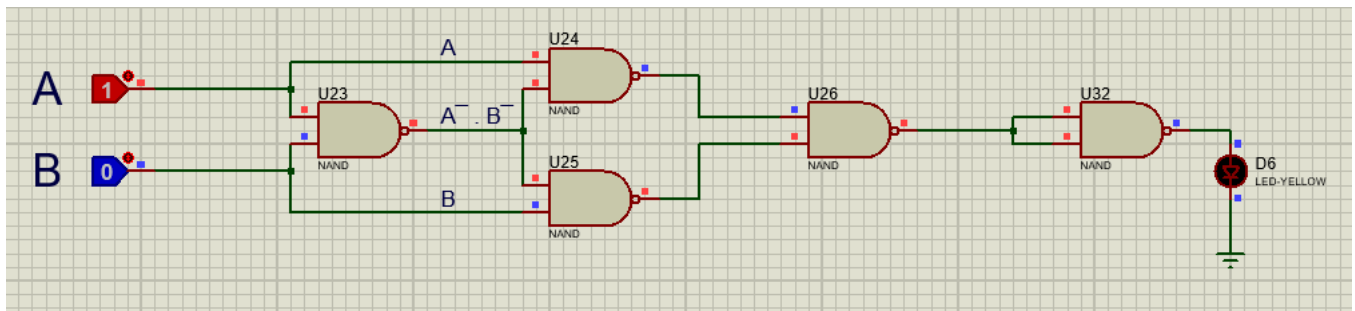
**Input: A=0, B=0 → Output=1**



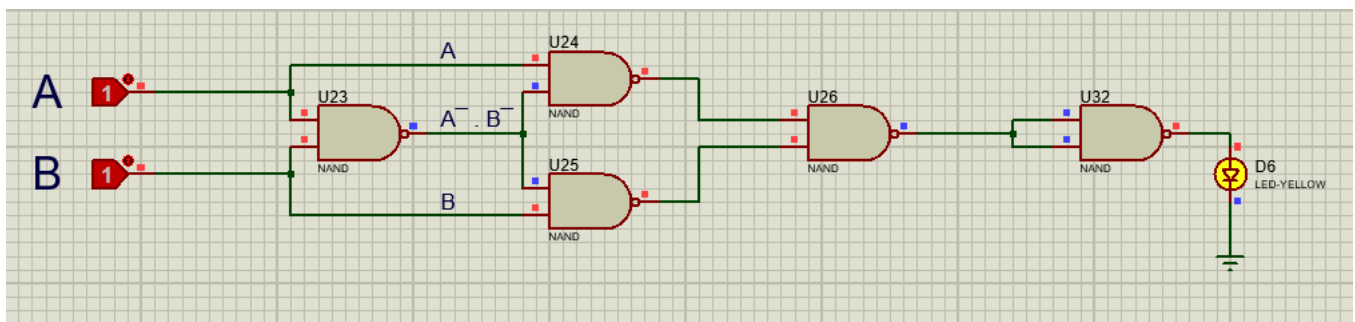
**Input: A=0, B=1 → Output=0**



**Input: A=1, B=0 → Output=0**



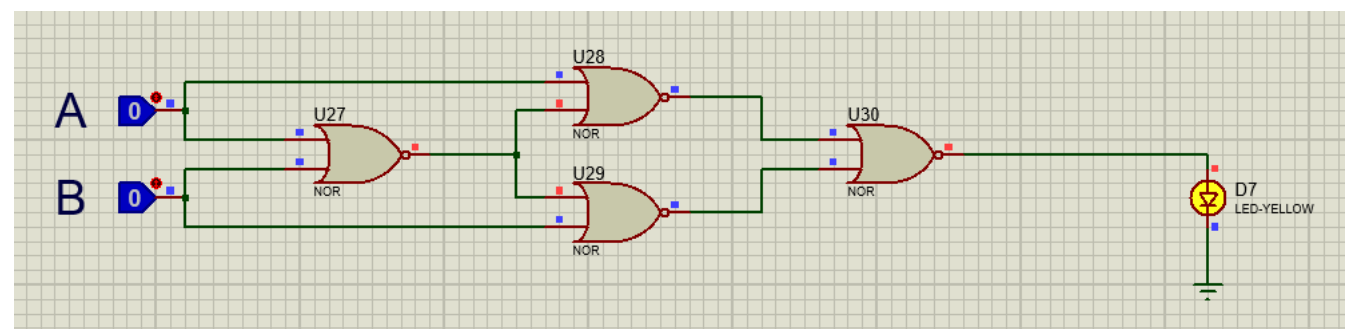
**Input: A=1, B=1 → Output=1**



## ii. XNOR using NOR gate

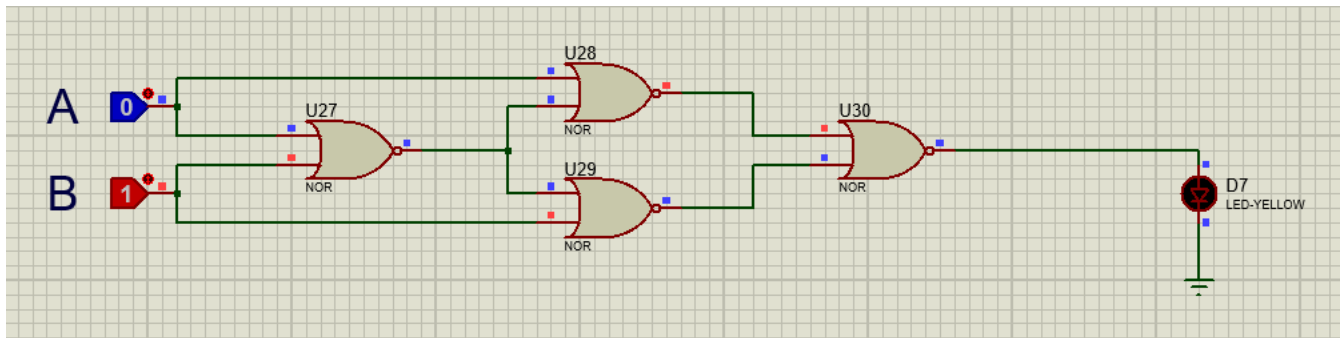
- Use a NOR gate to combine the inputs of A and B.
- Use second NOR gate to combine the output of first NOR gate and A.
- Use third NOR gate to combine the output of first NOR gate and B.
- Use fourth NOR gate to combine the output of second and third NOR gate for final output.

**Input: A=0, B=0 → Output=1**

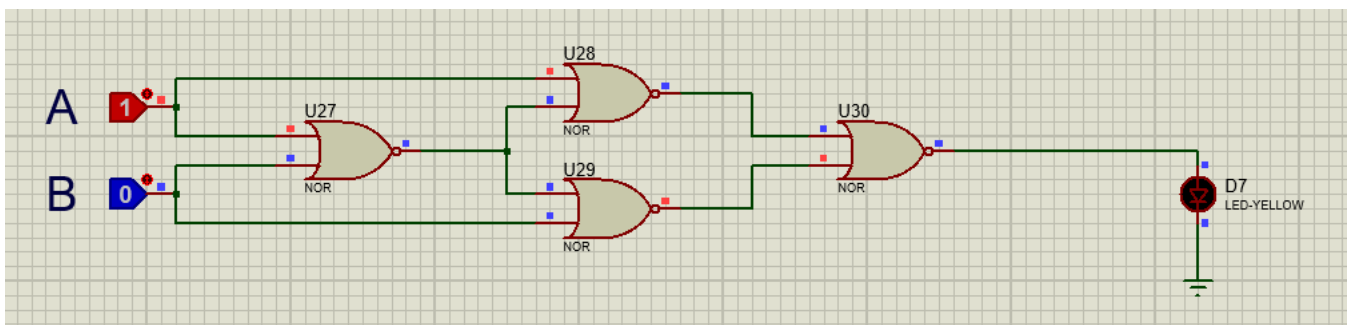




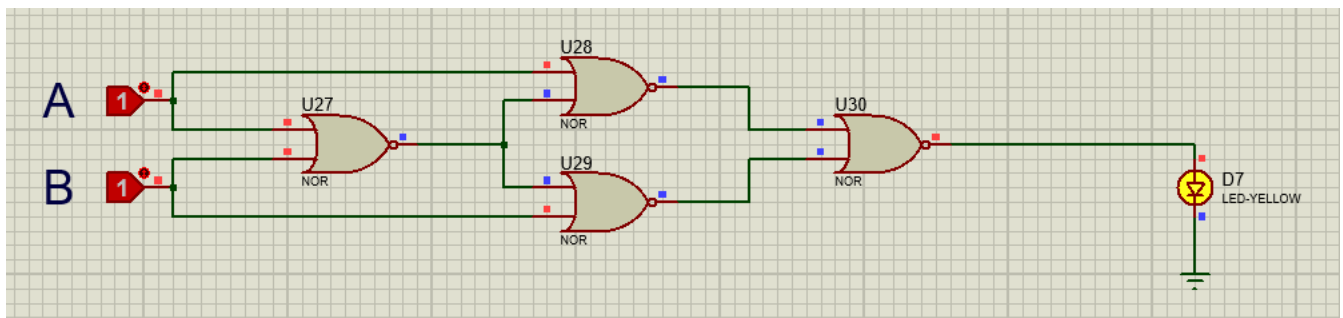
**Input: A=0, B=1 → Output=0**



**Input: A=1, B=0 → Output=0**



**Input: A=1, B=1 → Output=1**



**THE END**